

AI-ASSISTED CODING

Roll Number:2403A51457

AI-End LAb Exam

SET-6

Q1. AI Code Completion (Scenario)

A finance company needs an

`AccountLedger`

class to compute:

- deposits
- withdrawals
- running balance

Use AI-assisted completion to finish the class.

(Include documentation + comments + test cases.)

PROMPT:

Create a Finance Company needs an `AccountLedger` class to compute:

- deposits
- withdrawals
- running balance

INPUT:

```
import datetime

class AccountLedger:
    """
    Manages financial transactions (deposits and withdrawals) for an account
    and computes the running balance.
    """

    def __init__(self):
        """
        Initializes a new AccountLedger instance.
        Transactions are stored as a list of dictionaries.
        Each transaction dictionary will contain: 'type', 'amount', 'date'.
        """
        self.transactions = []
        print("AccountLedger initialized.")

    def deposit(self, amount):
        """
        Records a deposit transaction.

        Args:
            amount (float or int): The amount to be deposited. Must be positive.

        Raises:
            ValueError: If the amount is not positive.
        """
        if not isinstance(amount, (int, float)) or amount <= 0:
            raise ValueError("Deposit amount must be a positive number.")

        transaction = {
            'type': 'deposit',
            'amount': amount,
            'date': datetime.datetime.now()
        }
        self.transactions.append(transaction)
        print(f"Deposited: ${amount:.2f}")
```

```
# Test Case 1: Basic deposits and withdrawals
print("--- Test Case 1: Basic deposits and withdrawals ---")
ledger1 = AccountLedger()
ledger1.deposit(1000)
ledger1.withdraw(200)
ledger1.deposit(500)

running_balance_data = ledger1.get_running_balance()
print("Running Balance:")
for record in running_balance_data:
    print(f"Type: {record['type']}, Amount: ${record['amount']:.2f}, Balance: ${record['running_balance']:.2f}, Date: {record['date'].strftime('%Y-%m-%d %H:%M:%S')}")

# Verify final balance
expected_final_balance = 1000 - 200 + 500
actual_final_balance = running_balance_data[-1]['running_balance'] if running_balance_data else 0
print(f"Final Balance (expected: ${expected_final_balance:.2f}, Actual: ${actual_final_balance:.2f})")
assert actual_final_balance == expected_final_balance
print("Test Case 1 Passed!\n")

# Test Case 2: Attempting to withdraw more than available
print("--- Test Case 2: Insufficient funds ---")
ledger2 = AccountLedger()
ledger2.deposit(300)

try:
    ledger2.withdraw(500)
except InsufficientFundsError as e:
    print(f"Caught expected error: {e}")
    print("Test Case 2 Passed!\n")
except Exception as e:
    print(f"Caught unexpected error: {e}")
    print("Test Case 2 Failed!\n")

# Test Case 3: Ledger with no transactions
print("--- Test Case 3: Empty ledger ---")
ledger3 = AccountLedger()
running_balance_data_empty = ledger3.get_running_balance()
```

OUTPUT:

```
--- Test Case 1: Basic deposits and withdrawals ---
AccountLedger initialized.
Deposited: $1000.00
Withdrew: $200.00
Deposited: $500.00

Running Balance:
Type: deposit, Amount: $1000.00, Balance: $1000.00, Date: 2025-11-21 10:12:04
Type: withdrawal, Amount: $200.00, Balance: $800.00, Date: 2025-11-21 10:12:04
Type: deposit, Amount: $500.00, Balance: $1300.00, Date: 2025-11-21 10:12:04

Final Balance (Expected: $1300.00, Actual: $1300.00)
Test Case 1 Passed!

--- Test Case 2: Insufficient funds ---
AccountLedger initialized.
Deposited: $300.00
Caught expected error: Cannot withdraw $500.00. Insufficient funds. Current balance: $300.00
Test Case 2 Passed!

--- Test Case 3: Empty ledger ---
AccountLedger initialized.
Running Balance (Empty Ledger): []
Test Case 3 Passed!

--- Test Case 4: Multiple withdrawals ---
AccountLedger initialized.
Deposited: $1000.00
Withdrew: $100.00
Withdrew: $50.00
Withdrew: $200.00

Final Balance (Expected: $650.00, Actual: $650.00)
Test Case 4 Passed!

--- Test Case 5: Invalid amounts ---
AccountLedger initialized.
Caught expected error for deposit: Deposit amount must be a positive number.
Caught expected error for withdrawal: Withdrawal amount must be a positive number.
Test Case 5 Passed!
```

Q2. Algorithms (Scenario)

A document search tool needs both

Sequential Search

and

Fibonacci Search

implementations.

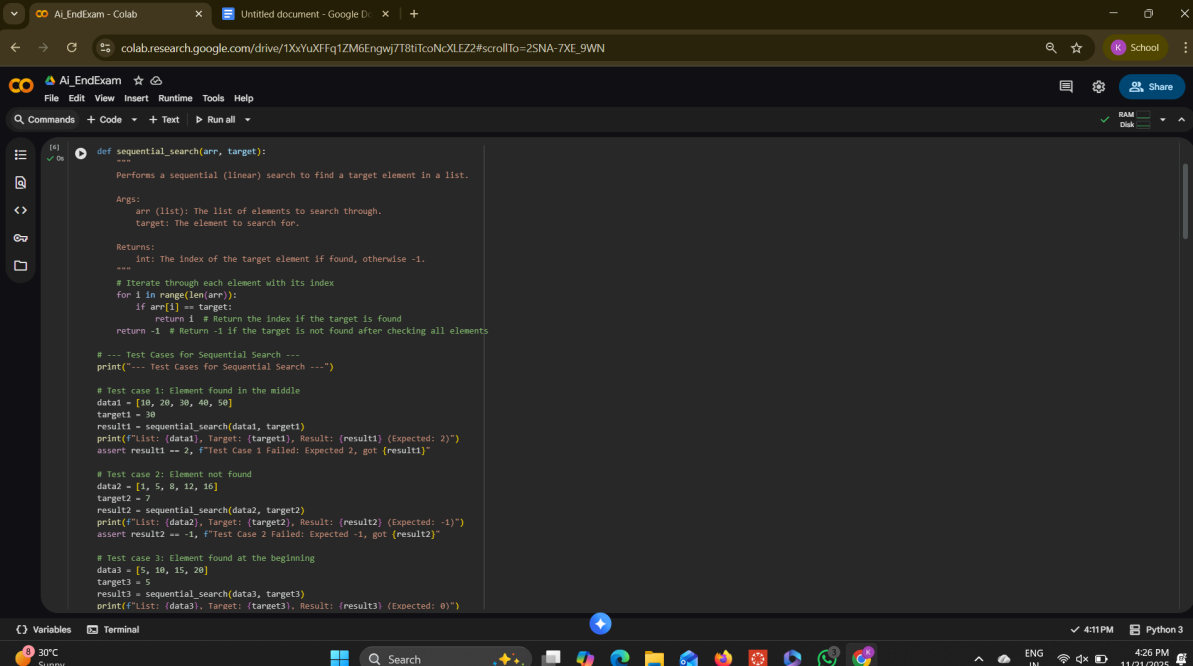
(Include documentation + examples + tests.)

PROMPT:

Create a Document search tool needs both Sequential Search and Fibonacci Search Implementations

INPUT:

For Sequential Search:



The screenshot shows a Google Colab notebook titled 'AI_EndExam'. The code defines a function `sequential_search(arr, target)` that performs a linear search. It includes docstrings for arguments and returns, and three test cases. The first test case finds the target 30 at index 2. The second test case does not find the target 7, returning -1. The third test case finds the target 5 at index 0. The output of the notebook shows the results of these tests.

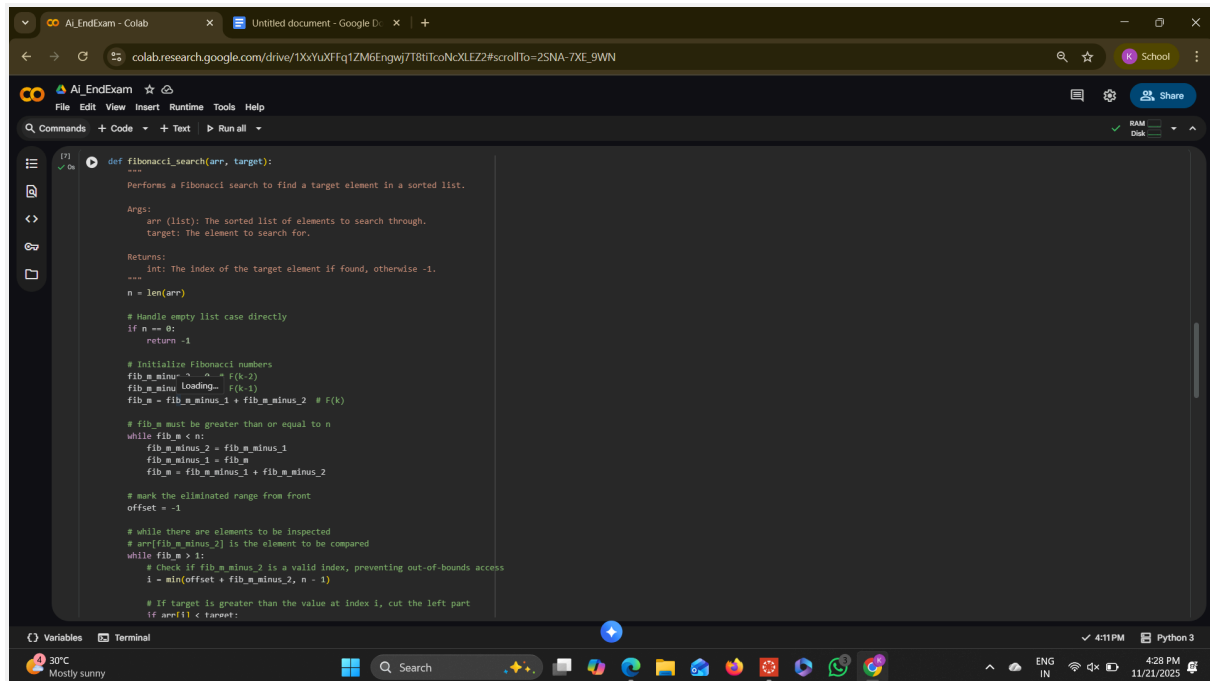
```
def sequential_search(arr, target):  
    """  
    Performs a sequential (linear) search to find a target element in a list.  
  
    Args:  
        arr (list): The list of elements to search through.  
        target: The element to search for.  
  
    Returns:  
        int: The index of the target element if found, otherwise -1.  
    """  
    # Iterate through each element with its index  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i # Return the index if the target is found  
    return -1 # Return -1 if the target is not found after checking all elements  
  
# --- Test Cases for Sequential Search ---  
print("--- Test Cases for Sequential Search ---")  
  
# Test case 1: Element found in the middle  
data1 = [10, 20, 30, 40, 50]  
target1 = 30  
result1 = sequential_search(data1, target1)  
print(f"List: {data1}, Target: {target1}, Result: {result1} (Expected: 2)")  
assert result1 == 2, f"Test Case 1 Failed: Expected 2, got {result1}"  
  
# Test case 2: Element not found  
data2 = [1, 5, 8, 12, 16]  
target2 = 7  
result2 = sequential_search(data2, target2)  
print(f"List: {data2}, Target: {target2}, Result: {result2} (Expected: -1)")  
assert result2 == -1, f"Test Case 2 Failed: Expected -1, got {result2}"  
  
# Test case 3: Element found at the beginning  
data3 = [5, 10, 15, 20]  
target3 = 5  
result3 = sequential_search(data3, target3)  
print(f"List: {data3}, Target: {target3}, Result: {result3} (Expected: 0)")
```

OUTPUT:

```
--- --- Test Cases for Sequential Search ---  
List: [10, 20, 30, 40, 50], Target: 30, Result: 2 (Expected: 2)  
List: [1, 5, 8, 12, 16], Target: 7, Result: -1 (Expected: -1)  
List: [5, 10, 15, 20], Target: 5, Result: 0 (Expected: 0)  
List: [100, 200, 300, 400], Target: 400, Result: 3 (Expected: 3)  
List: [], Target: 10, Result: -1 (Expected: -1)  
All Sequential Search test cases passed!
```

INPUT:

For Fibonacci Search:



The screenshot shows a Google Colab notebook with a Python function named `fibonacci_search`. The function takes an array `arr` and a target value `target` as input. It implements the Fibonacci search algorithm by generating Fibonacci numbers and using them to divide the array into segments. The function returns the index of the target element if found, or -1 otherwise. The notebook interface includes a file explorer, a command palette, and a terminal at the bottom.

```
[?] def fibonacci_search(arr, target):  
    """  
    Performs a Fibonacci search to find a target element in a sorted list.  
    ---  
    Args:  
        arr (list): The sorted list of elements to search through.  
        target: The element to search for.  
    Returns:  
        int: The index of the target element if found, otherwise -1.  
    ---  
    n = len(arr)  
    # Handle empty list case directly  
    if n == 0:  
        return -1  
    # Initialize Fibonacci numbers  
    fib_m_minus_2 = 0 # f(k-2)  
    fib_m_minus_1 = 1 # f(k-1)  
    fib_m = fib_m_minus_1 + fib_m_minus_2 # f(k)  
    # fib_m must be greater than or equal to n  
    while fib_m < n:  
        fib_m_minus_2 = fib_m_minus_1  
        fib_m_minus_1 = fib_m  
        fib_m = fib_m_minus_1 + fib_m_minus_2  
    # mark the eliminated range from front  
    offset = -1  
    # while there are elements to be inspected  
    # arr[fib_m_minus_2] is the element to be compared  
    while fib_m > 1:  
        # check if fib_m_minus_2 is a valid index, preventing out-of-bounds access  
        i = min(offset + fib_m_minus_2, n - 1)  
        # if target is greater than the value at index i, cut the left part  
        if arr[i] < target:
```

OUTPUT:

```
... --- Test Cases for Fibonacci Search ---  
List: [10, 20, 30, 40, 50, 60, 70, 80, 90], Target: 50, Result: 4 (Expected: 4)  
List: [1, 5, 8, 12, 16, 20], Target: 7, Result: -1 (Expected: -1)  
List: [5, 10, 15, 20, 25], Target: 5, Result: 0 (Expected: 0)  
List: [100, 200, 300, 400, 500], Target: 500, Result: 4 (Expected: 4)  
List: [], Target: 10, Result: -1 (Expected: -1)  
List: [42], Target: 42, Result: 0 (Expected: 0)  
List: [42], Target: 99, Result: -1 (Expected: -1)  
All Fibonacci Search test cases passed!
```